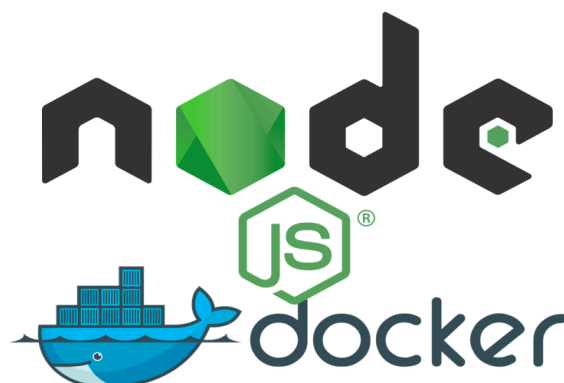


Container con Codespace e Docker

- Esercizio A -	2
Container nodejs con GitHub Codespaces	2
Descrizione del Problema	2
Implementazione	2
Collaudo	2
Procedimento e Risultati	6
Considerazioni Finali	7
- Esercizio B -	8
Gestione di container Docker	8
Descrizione del Problema	8
Implementazione	8
Collaudo	8
Procedimento e Risultati	17
Considerazioni Finali	17
- Esercizio C -	18
Agente AI con Agno, Mistral e Telegram	18
Descrizione del Problema	18
Implementazione	18
Collaudo	18
Procedimento e Risultati	21
Domande di riflessione	22
Considerazioni Finali	23



**DEVELOP & DEPLOY NODE JS WEB APPLICATION
IN DOCKER**

- Esercizio A -

Container nodejs con GitHub Codespaces

Descrizione del Problema

La richiesta introduce l'uso di Dev Container in GitHub Codespaces per creare ambienti di sviluppo riproducibili. Il problema affrontato consiste nel configurare da zero un container di sviluppo (con Node.js e Docker-in-Docker), avviare un ambiente preconfigurato da un repository esistente, e infine sviluppare ed eseguire una semplice API REST in Node.js/Express all'interno del container, testandone il funzionamento. Si confronta così lo sviluppo diretto nel Dev Container con l'approccio basato su Docker, gettando le basi per la containerizzazione di applicazioni cloud.

Implementazione

◆ Laboratorio guidato

Collaudo

```
MINGW64:/c/Users/feded/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto/il-mio-lab
feded@LAPTOP-M4VVVMR2 MINGW64 ~
$ cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace&Docker\EsercizioA\Progetto
[1] 1595
bash: cd: C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace: No such file or directory
bash: DockerEsercizioAProgetto: command not found
[1]+  Exit 1                  cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace

feded@LAPTOP-M4VVVMR2 MINGW64 ~
$ cd "C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace&Docker\EsercizioA\Progetto"

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto (main)
$ mkdir il-mio-lab

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto (main)
$ cd il-mio-lab

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto/il-mio-lab (main)
$ git init
Initialized empty Git repository in C:/Users/feded/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto/il-mio-lab/.git/
feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto/il-mio-lab (master)
$ |
```

Sul terminale Git Bash, si naviga fino alla cartella di progetto e si esegue mkdir 'il-mio-lab' per creare la directory di lavoro. Successivamente si entra nella cartella e si lancia git init per inizializzare un repository Git locale, gettando le basi per il versionamento del codice e della configurazione del Dev Container.

```
MINGW64:/c/Users/feded/Desktop/FileSistemi/Esercitazione_6-Codespace&D...
GNU nano 8.7 .devcontainer/devcontainer.json
{
  "name": "Il mio Lab",
  "image": "mcr.microsoft.com/devcontainers/base:debian",

  "features": {
    // Installa Node.js versione 24
    "ghcr.io/devcontainers/features/node:1": { "version": "24" },

    // Installa Docker-in-Docker (per usare docker dentro il container)
    "ghcr.io/devcontainers/features/docker-in-docker:2": { "moby": false }
  },

  // Comando eseguito UNA VOLTA dopo la creazione del container
  "postCreateCommand": "docker --version && node --version",

  // Porte da esporre automaticamente verso il browser
  "forwardPorts": [3000, 8080, 8888]
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Usando l'editor nano viene creato il file '.devcontainer/devcontainer.json'. Configurazione specifica: nome del container ("Il mio Lab"), immagine base Debian ufficiale, due features (Node.js versione 24 e Docker-in-Docker con moby disabilitato), un comando post-creazione che mostra le versioni di Docker e Node, e le porte 3000, 8080, 8888 da inoltrare automaticamente.

```
feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (master)
$ git add .devcontainer/devcontainer.json

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (master)
$ git commit -m "feat: add devcontainer configuration"
[master (root-commit) 5489b4a] feat: add devcontainer configuration
1 file changed, 18 insertions(+)
create mode 100644 .devcontainer/devcontainer.json

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (master)
$
```

Dopo aver aggiunto il file con `git add .devcontainer/devcontainer.json`, l'utente esegue `git commit -m "feat: add devcontainer configuration"`. Il commit registra la creazione del file (18 righe inserite) con hash 5489b4a, rendendo persistente la configurazione del Dev Container all'interno del repository locale.

The screenshot shows the VS Code interface with the `devcontainer.json` file open in the Explorer. The file contains the following configuration:

```
1 {
2   "name": "il mio Lab",
3   "image": "mcr.microsoft.com/devcontainers/base:debian",
4
5   "features": {
6     // Installa Node.js versione 24
7     "ghcr.io/devcontainers/features/node:1": { "version": "24" },
8
9     // Installa Docker-in-Docker (per usare docker dentro il container)
10    "ghcr.io/devcontainers/features/docker-in-docker:2": { "moby": false }
11  },
12
13  // Comando eseguito UNA VOLTA dopo la creazione del container
14  "postCreateCommand": "docker --version && node --version",
15
16  // Porte da esporre automaticamente verso il browser
17  "forwardPorts": [3000, 8080, 8888]
18 }
19
```

The integrated terminal at the bottom shows the output of the `postCreateCommand`:

```
bash - il-mio-lab
vscode → /workspaces/il-mio-lab $ node --version
v24.15.0
vscode → /workspaces/il-mio-lab $ docker --version
Docker version 29.4.1, build 055a478
vscode → /workspaces/il-mio-lab $
```

Nell'ambiente VS Code si visualizza il file `devcontainer.json` aperto. Nel terminale integrato si vede l'output locale: `node --version` restituisce `v24.15.0` e `docker --version` restituisce `Docker version 29.4.1`, confermando che Node.js e Docker-in-Docker sono stati installati correttamente nel container.

Port	Forwarded Address	Running Process	Origin
3000	localhost:3000		Dev Containers
8080	localhost:8080		Dev Containers
8888	localhost:8888		Dev Containers

[Add Port](#)

Dalla sezione PORTS di VS Code si osserva che le tre porte definite (3000, 8080, 8888) sono state automaticamente inoltrate verso localhost dal Dev Container. Ogni porta è associata all'origine Dev Containers e risulta pronta per essere utilizzata da eventuali applicazioni web o API in esecuzione all'interno del container, senza bisogno di configurazioni aggiuntive. (È opportuno specificare come la prima parte dell'esercitazione sia stata eseguita in locale, mentre la successiva all'interno dell'ambiente Github Codespaces. Ciò ha permesso di testare sia una modalità di programmazione locale che remota).

```
PROBLEMI  OUTPUT  CONSOLE DI DEBUG  TERMINALE  PORTE

• @eFFe0513 → /workspaces/cloud-computing-lab (main) $ node --version
v24.14.0
• @eFFe0513 → /workspaces/cloud-computing-lab (main) $ docker --version
Docker version 29.3.0-1, build 5927d80c76b3ce5cf782be818922966e8a0d87a3
○ @eFFe0513 → /workspaces/cloud-computing-lab (main) $
```

Nel terminale del Codespace aperto sul repository `cloud-computing-lab`, l'utente lancia `node --version` e `docker --version`. I risultati mostrano Node.js `v24.14.0` e Docker `29.3.0-1`, confermando che l'ambiente di sviluppo containerizzato è già configurato correttamente con gli strumenti necessari, senza bisogno di installazioni aggiuntive.

```

• @eFFe0513 →/workspaces/cloud-computing-lab (main) $ mkdir nodejs-app
• @eFFe0513 →/workspaces/cloud-computing-lab (main) $ cd nodejs-app
• @eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $ npm init -y
Wrote to /workspaces/cloud-computing-lab/nodejs-app/package.json:

{
  "name": "nodejs-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

```

Il prompt del terminale indica che l'utente si trova all'interno della cartella /workspaces/cloud-computing-lab (il repository clonato automaticamente nel Codespace). L'immagine mostra visivamente il percorso completo, evidenziando che si sta operando nella root del progetto, pronti per spostarsi nella sotto-cartella nodejs-app dove verrà scritta l'applicazione.

```

• @eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $ npm install express

added 65 packages, and audited 66 packages in 3s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
• @eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $ cat > server.js << 'EOF'
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.json({
    message: 'Hello from Node.js!Federico',
    service: 'nodejs-api'
  });
});

app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`✅ Node.js app running on port ${PORT}`);
});
EOF
• @eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $

```

L'utente si sposta in nodejs-app ed esegue `npm install express` per installare il framework (vengono aggiunti 65 pacchetti). Successivamente, tramite `cat > server.js << 'EOF' ... EOF`, crea il file `server.js` con il codice: due endpoint (`/` che restituisce un messaggio personalizzato "Hello from Node.js!Federico" e `/health` che restituisce `{"status":"ok"}`) e il server in ascolto sulla porta 3000.

```
@eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $ npm start
Debugger attached.

> nodejs-app@1.0.0 start
> node server.js

Debugger attached.
✓ Node.js app running on port 3000
```

Viene lanciato il comando `npm start` (definito nel `package.json` come `node server.js`). Il terminale mostra l'output `Debugger attached` (dovuto al debugger di VS Code) e il messaggio `Node.js app running on port 3000`, indicando che il server Express è partito correttamente e rimane in esecuzione in attesa di richieste.

```
● @eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $ curl http://localhost:3000/
{"message":"Hello from Node.js!Federico","service":"nodejs-api"}
● @eFFe0513 →/workspaces/cloud-computing-lab/nodejs-app (main) $ curl http://localhost:3000/health
{"status":"ok"}
○ s-app (main) $
```

Aperto un nuovo terminale ed eseguendo `curl http://localhost:3000/`, si ottiene la risposta JSON `{"message":"Hello from Node.js!Federico","service":"nodejs-api"}`. Successivamente testando l'endpoint di health con `curl http://localhost:3000/health`, si riceve `{"status":"ok"}`. Entrambe le chiamate hanno successo, verificando il corretto funzionamento dell'applicazione all'interno del Dev Container.

Procedimento e Risultati

L'esercitazione è iniziata con la creazione manuale di un Dev Container. Dopo aver preparato una cartella locale e inizializzato un repository Git, è stato realizzato il file `.devcontainer/devcontainer.json` che specifica un'immagine base Debian, l'installazione di Node.js 24 e Docker-in-Docker, e l'inoltro automatico delle porte 3000, 8080 e 8888. Nel terminale integrato vs locale è apparsa la conferma delle versioni: Node.js v24.15.0 e Docker 29.4.1, e le porte erano già inoltrate su localhost. Si è poi passati a un repository già configurato, `cloud-computing-lab`, dove sono state verificate le medesime condizioni. All'interno della cartella `nodejs-app` è stato

installato Express e creato un semplice server.js con due endpoint, uno sulla radice che restituisce un messaggio personalizzato e uno su /health che restituisce {"status":"ok"}. L'avvio con npm start ha fatto partire il server sulla porta 3000 e i test con curl hanno dato esito positivo, confermando il pieno funzionamento dell'applicazione all'interno del Dev Container.

Considerazioni Finali

Questa esercitazione ha mostrato concretamente i vantaggi dei Dev Container in GitHub Codespaces. Configurare manualmente il container ha richiesto pochi passaggi ma ha offerto un controllo completo sull'ambiente, mentre l'uso di un repository preconfigurato ha dimostrato la riproducibilità immediata. L'inoltro automatico delle porte e la presenza di Docker-in-Docker semplificano notevolmente lo sviluppo e il test di applicazioni. In conclusione, i Dev Container rappresentano uno strumento efficace per il cloud computing, garantendo ambienti isolati e pronti all'uso, riducendo i problemi di compatibilità e facilitando il lavoro collaborativo.

- Esercizio B -

Gestione di container Docker

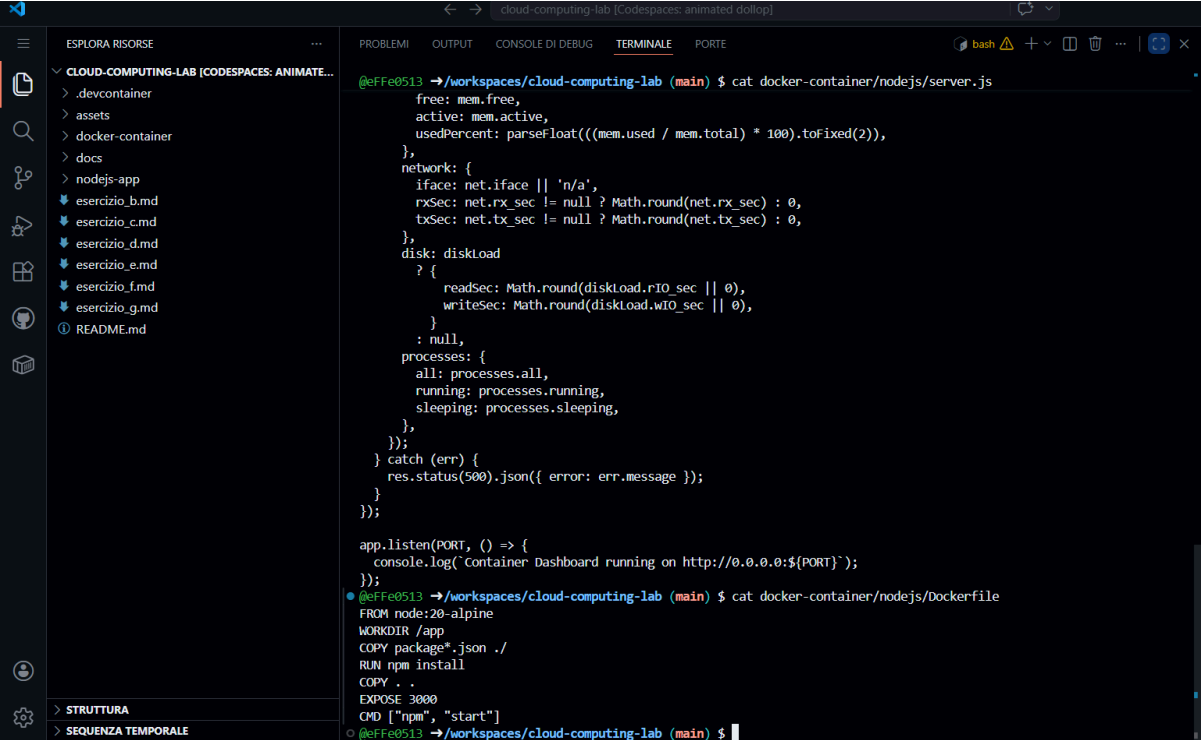
Descrizione del Problema

In questo elaborato si affronta la containerizzazione di applicazioni eterogenee tramite Docker in ambiente GitHub Codespaces. Dopo fork e clonazione del repository, si avviano tre stack: un'API REST in Node.js, un'applicazione Java con Spring Boot e un sistema LAMP orchestrato con docker compose. L'attività sviluppa la padronanza dei comandi 'docker run' e 'docker compose', delle multi-stage build e della configurazione di servizi con variabili d'ambiente, volumi e healthcheck. Il test funzionale delle API, eseguito via browser e 'curl' direttamente nel Codespace, completa il percorso, offrendo un'esperienza pratica di gestione container in un contesto cloud riproducibile.

Implementazione

◆ Laboratorio guidato

Collaudo



```
cloud-computing-lab [Codespaces: animated dollop]

ESPLORA RISORSE
CLOUD-COMPUTING-LAB [CODESPACES: ANIMATE...
  > .devcontainer
  > assets
  > docker-container
  > docs
  > nodejs-app
  > esercizio_b.md
  > esercizio_c.md
  > esercizio_d.md
  > esercizio_e.md
  > esercizio_f.md
  > esercizio_g.md
  > README.md

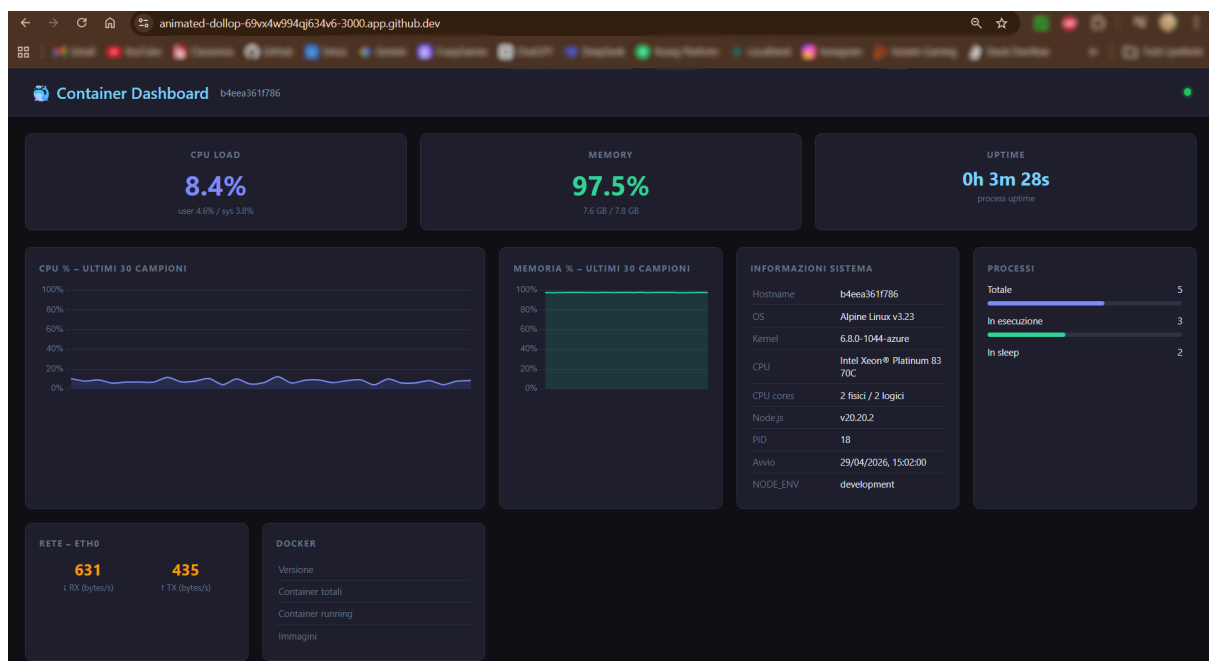
PROBLEMI OUTPUT CONSOLE DI DEBUG TERMINALE PORTE
@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cat docker-container/nodejs/server.js
    free: mem.free,
    active: mem.active,
    usedPercent: parseFloat(((mem.used / mem.total) * 100).toFixed(2)),
  },
  network: {
    iface: net.iface || 'n/a',
    rxSec: net.rx_sec != null ? Math.round(net.rx_sec) : 0,
    txSec: net.tx_sec != null ? Math.round(net.tx_sec) : 0,
  },
  disk: diskLoad
    ? {
        readSec: Math.round(diskLoad.rIO_sec || 0),
        writeSec: Math.round(diskLoad.wIO_sec || 0),
      }
    : null,
  processes: {
    all: processes.all,
    running: processes.running,
    sleeping: processes.sleeping,
  },
});
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

app.listen(PORT, () => {
  console.log('Container Dashboard running on http://0.0.0.0:${PORT}');
});
@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cat docker-container/nodejs/Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
@eFFe0513 →/workspaces/cloud-computing-lab (main) $
```

Viene mostrato il codice del server Node.js, che espone endpoint per il monitoraggio in tempo reale di CPU, memoria, processi e rete, e il Dockerfile corrispondente basato su node:20-alpine, con copia dei sorgenti e avvio tramite npm start.

```
@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cd docker-container/nodejs
docker build -t nodejs-api:1.0 .
[+] Building 9.8s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 150B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  1.2s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f6  2.2s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f6  0.0s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122  1.26MB / 1.26MB  0.2s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096  445B / 445B  0.2s
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287  43.23MB / 43.23MB  1.1s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb  3.86MB / 3.86MB  0.4s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb  0.2s
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287  1.0s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122  0.0s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096  0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 21.80kB                                   0.0s
=> [2/5] WORKDIR /app                                           1.2s
=> [3/5] COPY package*.json ./                                  0.2s
=> [4/5] RUN npm install                                         4.0s
=> [5/5] COPY . .                                               0.1s
=> exporting to image                                           0.8s
=> => exporting layers                                             0.5s
=> => exporting manifest sha256:7999fe2262d3e9a19b07d2980252d94937b4767ba0a04bffa5626b0da6439012  0.0s
=> => exporting config sha256:d6c2ff6f57ea0d59fc59e938478fdc3e024885227fb42513dda527326865b8b  0.0s
=> => exporting attestation manifest sha256:57085eb3076f09832414bd5567a8a4c15bf7ad6010d1e34a42aea826ccd55dbe  0.0s
=> => exporting manifest list sha256:86213fefbffa474288acbc90086c3d90dba5fb3d67834d354c42a3b0b669206c8  0.0s
=> => naming to docker.io/library/nodejs-api:1.0                 0.0s
=> => unpacking to docker.io/library/nodejs-api:1.0               0.2s
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/nodejs (main) $
```

La build dell'immagine node js-api:1.0 evidenzia i passaggi del Dockerfile: dalla risoluzione dell'immagine alpine, all'installazione dei pacchetti npm, fino alla creazione e all'esportazione dell'immagine finale.



La dashboard web visualizza le metriche di sistema del container in esecuzione: carico CPU, utilizzo della memoria, uptime, informazioni sull'host (Alpine, kernel, CPU) e stato dei processi, offrendo un monitoraggio immediato delle risorse.

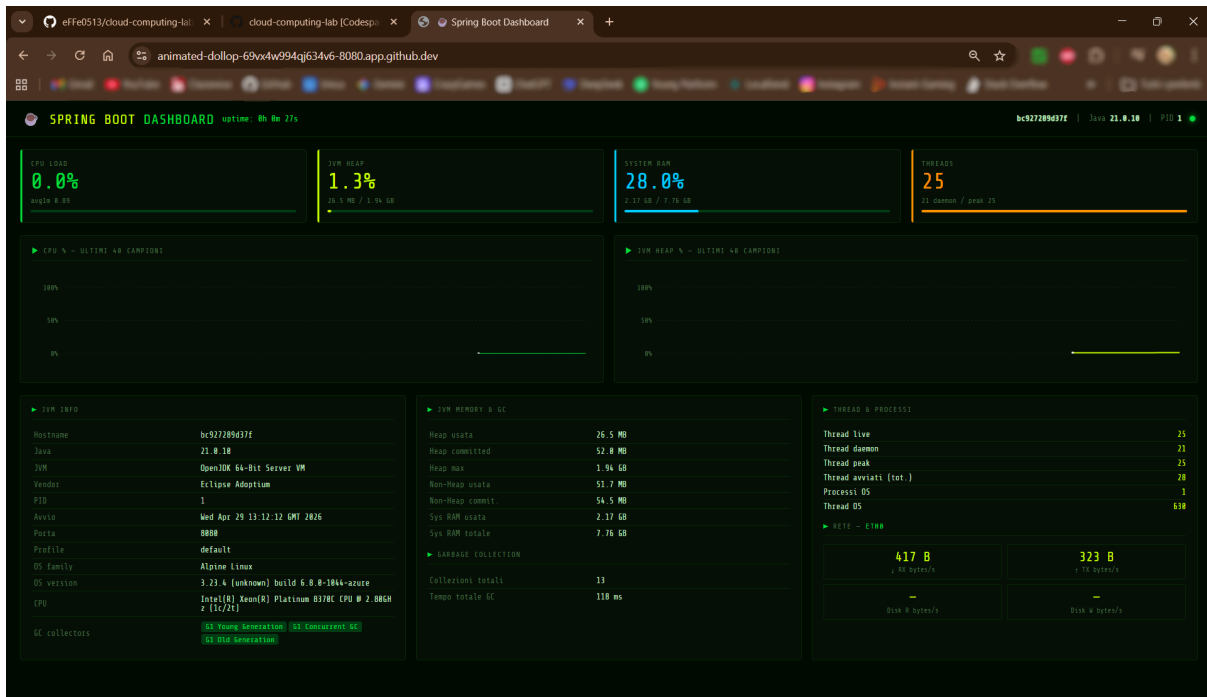
```
@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cat docker-container/java-spring/Dockerfile
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Il Dockerfile per l'applicazione Java Spring adotta una strategia multi-stage: un primo stadio con Maven e JDK 21 compila il progetto, il secondo stadio con sola JRE 21 Alpine copia il jar e lo esegue, riducendo drasticamente le dimensioni dell'immagine.

```
@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cd docker-container/java-spring
docker build -t spring-dashboard:1.0 .
=> => extracting sha256:b40150c1c2717d324cdb17278c8efdfa4dfcd2ffe083e976f0bcedf31115f081 2.9s
=> => extracting sha256:a2c74f5c2809456d4ea1d8df7e9a8c6ce210cd1a451308fd3ee42cd5613fac2f 3.1s
=> => extracting sha256:3b9a1e97487bbf567fd39c1a971160f21dc69bfea8eeb8bfc923b39fe9c05f8d 4.9s
=> => extracting sha256:2ef33ef1b9930d5fb66257dbd8f7e1ccb258d35b87243481902922fcdcaefae86 0.0s
=> => extracting sha256:e9d29f6a4c60335d154318f22c76be42011017740ad6c05fdb6d56ee283f3030 0.0s
=> => extracting sha256:3a0cfe6cd7e3ea307af3c977e7587353fbdde9c79b6195c14f68ff7e9f914d84 0.7s
=> => extracting sha256:c708c0a14e8a983cac9b61f9722467ff85661198c365529d9f54f54bc61cbeee 0.3s
=> => extracting sha256:b552d4a3844f33860c2d9492a88e3c76a8135878a304a0971b888144bda2e33b 0.0s
=> => extracting sha256:1396908a28e98cde78ac7aec848b953c7b0f1c2ccdc64efa4eec120b5c8f6f44 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 31.82kB 0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:ad0cdd9782db550ca7dde6939a16fd850 7.1s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:ad0cdd9782db550ca7dde6939a16fd850d04e683d 0.0s
=> => sha256:48e51247df3167f00ff9972116a308eb551b96eac8382fbf88e82d5866971c 2.28kB / 2.28kB 0.1s
=> => sha256:7659027a798a9c8fca5bfcbae2d24bd6b7ec65dfe5285d4b7c30c75ca0965299 128B / 128B 0.2s
=> => sha256:83cc7558c41737fe273044d032b589323a9acd96790bbfd6cf2792647fb88e3b 53.17MB / 53.17MB 1.6s
=> => sha256:775c53a23a895a8fdbc2534de50a64b420d676e982f81a1e72e88f6b283280fe 16.84MB / 16.84MB 0.7s
=> => extracting sha256:775c53a23a895a8fdbc2534de50a64b420d676e982f81a1e72e88f6b283280fe 1.0s
=> => extracting sha256:83cc7558c41737fe273044d032b589323a9acd96790bbfd6cf2792647fb88e3b 5.1s
=> => extracting sha256:7659027a798a9c8fca5bfcbae2d24bd6b7ec65dfe5285d4b7c30c75ca0965299 0.0s
=> => extracting sha256:48e51247df3167f00ff9972116a308eb551b96eac8382fbf88e82d5866971c 0.0s
=> [stage-1 2/3] WORKDIR /app 1.5s
=> [builder 2/6] WORKDIR /app 2.1s
=> [builder 3/6] COPY pom.xml . 0.0s
=> [builder 4/6] RUN mvn dependency:go-offline 24.5s
=> [builder 5/6] COPY src ./src 0.1s
=> [builder 6/6] RUN mvn package -DskipTests 6.8s
=> [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar 0.3s
=> exporting to image 1.3s
=> => exporting layers 1.1s
=> => exporting manifest sha256:c5f9ed6d3dda483ec5ebb6b084fb917f7c9a857057e5a27b406626bb78b8e734 0.0s
=> => exporting config sha256:0e658f0fe09a0943e29c12ca6ba8a184a057b97e7cc126a0db32aef28400e8f4 0.0s
=> => exporting attestation manifest sha256:7546fa5a0147198514621733766b0ebc0c0e31754cefd550c9894aee918e4397 0.0s
=> => exporting manifest list sha256:1ca30f3cc5225e6446117d8f96d9aaf4ce74b392088dbc9ad324fd9dfdf8028c9 0.0s
=> => naming to docker.io/library/spring-dashboard:1.0 0.0s
=> => unpacking to docker.io/library/spring-dashboard:1.0 0.2s
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/java-spring (main) $
```

L'output della build dell'immagine Spring mostra i due stadi in azione: la risoluzione delle dipendenze Maven, la compilazione, e infine la creazione dell'immagine finale a partire da eclipse-temurin:21-jre-alpine, pronta per l'esecuzione su porta 8080.



La dashboard Spring Boot esposta sulla porta 8080 presenta metriche in tempo reale del container: carico CPU, utilizzo heap JVM, informazioni sul sistema operativo Alpine, dettagli sulla garbage collection, thread e traffico di rete.

```
@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cat docker-container/lamp/Dockerfile
FROM php:8.2.5-apache-bullseye

# Install PDO MySQL and mysqli extensions
RUN docker-php-ext-install pdo_mysql mysqli
```

Il Dockerfile per il servizio LAMP parte dall'immagine php:8.2.5-apache-bullseye e installa le estensioni pdo_mysql e mysqli, abilitando la comunicazione tra PHP e il database MariaDB.

```

@eFFe0513 →/workspaces/cloud-computing-lab (main) $ cat docker-container/lamp/docker-compose.yml
services:
  mariadb:
    image: mariadb:10.11
    container_name: lamp_mariadb
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
    volumes:
      - ./volumes/mysql:/var/lib/mysql
    networks:
      - lamp_network
    healthcheck:
      test: ["CMD-SHELL", "mariadb-admin ping -h 127.0.0.1 -u root -p${MYSQL_ROOT_PASSWORD} --silent"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
      restart: unless-stopped

  webserver:
    build: .
    image: lamp_webserver:latest
    container_name: lamp_webserver
    depends_on:
      mariadb:
        condition: service_healthy
    environment:
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_HOST: ${DB_HOST}
      WEBSERVER_ADMIN_USERNAME: ${WEBSERVER_ADMIN_USERNAME}
      WEBSERVER_ADMIN_PASSWORD: ${WEBSERVER_ADMIN_PASSWORD}
    volumes:
      # Web content directories
      - ./volumes/www:/var/www/html/
      # Apache configuration

```

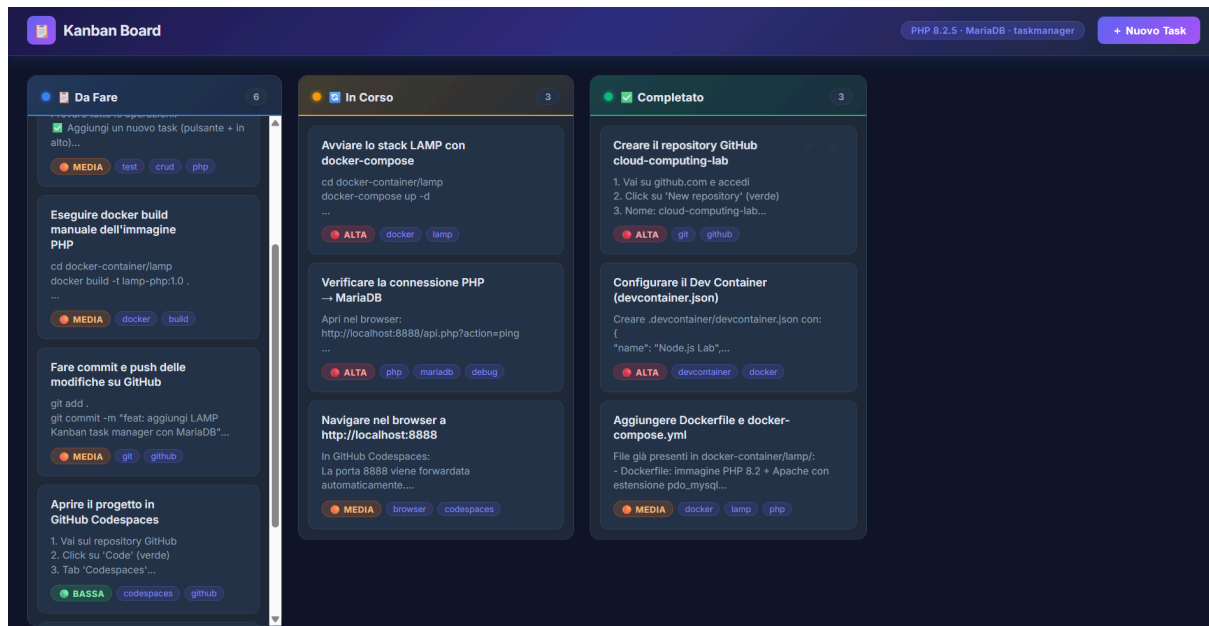
Lo stralcio del docker-compose.yml definisce i due servizi principali: mariadb, con credenziali via variabili d'ambiente, volume persistente e healthcheck tramite mariadb-admin ping, e webserver, che dipende dal database e monta i contenuti web e le configurazioni Apache.

```

@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $ docker compose up -d
=> => sha256:c388a546afa6d7bed91c288fbc838496d1c2b09c2f70465a5cd69032bfecf072 12.34MB / 12.34MB 0.5s
=> => sha256:1d02a81768ed9803772d4654c2b55b24c54d1200a872fcc3b689b8dd84cebd9 511B / 511B 0.4s
=> => sha256:bf1c5be6427ed0108e9859238ab41d37f43b792c65758e165f1cb957e7c4ab45 476B / 476B 1.5s
=> => sha256:8681ad2eeea6fa7d3aed97bcd471bddff6b4932f405ecb84452c3d2522390e2a 271B / 271B 1.5s
=> => sha256:92711ce78973323920b717b53e3a3e197c7b958c6e1a1be2a69b72718519c96a 19.25MB / 19.25MB 1.6s
=> => sha256:5908153120ba6d8988feadd5f6a8c68a281970ad001ac3be0f5bc9dc030b5c4d 91.64MB / 91.64MB 2.9s
=> => sha256:07353b772b5e325bf0abf03512cfaebe590a14a0858cf0ec2740b5ab055a31a7 225B / 225B 0.1s
=> => sha256:9e3ea8720c6de96cc9ad544dddc695a3ab73f5581c5d954e0504cc4f80fb5e5c 31.40MB / 31.40MB 1.3s
=> => extracting sha256:9e3ea8720c6de96cc9ad544dddc695a3ab73f5581c5d954e0504cc4f80fb5e5c 4.8s
=> => extracting sha256:07353b772b5e325bf0abf03512cfaebe590a14a0858cf0ec2740b5ab055a31a7 0.0s
=> => extracting sha256:5908153120ba6d8988feadd5f6a8c68a281970ad001ac3be0f5bc9dc030b5c4d 5.2s
=> => extracting sha256:8681ad2eeea6fa7d3aed97bcd471bddff6b4932f405ecb84452c3d2522390e2a 0.0s
=> => extracting sha256:92711ce78973323920b717b53e3a3e197c7b958c6e1a1be2a69b72718519c96a 0.4s
=> => extracting sha256:bf1c5be6427ed0108e9859238ab41d37f43b792c65758e165f1cb957e7c4ab45 0.0s
=> => extracting sha256:1d02a81768ed9803772d4654c2b55b24c54d1200a872fcc3b689b8dd84cebd9 0.0s
=> => extracting sha256:c388a546afa6d7bed91c288fbc838496d1c2b09c2f70465a5cd69032bfecf072 0.1s
=> => extracting sha256:f4af9cab66dfd5c1f054a5efd32dbe0f3dcea0b32f5143ab1ec6f2c3c3891cff 0.0s
=> => extracting sha256:8ad48037138e034a4441c3e0c124f6ad400668e52368bbc55c13a13b325fb17 0.3s
=> => extracting sha256:2da00dd3bf93deaf638bdc5490562258a4f48ffdb7b221ae5e9c1b04457cba5 0.0s
=> => extracting sha256:b137c31a4cadca4abbef99bb68f7c341edeb2135e457299fdb2793124ca614b 0.0s
=> => extracting sha256:e2f0d029c21f08a4852345f3390f040499a630cab3bb5418e08f78caa1a068a8 0.0s
=> [2/2] RUN docker-php-ext-install pdo_mysql mysqli 16.2s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:e32d2e2d23b4702428c39a647ab7fef771b195a494253d5a970fb3f051d9a5a4 0.0s
=> => exporting config sha256:24a5eafb86e5dc176f063fa610cadf348a834298c57ac944bc1109c1b18415f9 0.0s
=> => exporting attestation manifest sha256:1b8b6342eae4f43a0e4e9a0bb250986ecba4c778916074dccc6408dbe51f26d9 0.0s
=> => exporting manifest list sha256:e438755925127b6fdb232e9c116306912628e1874853d8d8100cbbdbca92f8cd 0.0s
=> => naming to docker.io/library/lamp_webserver:latest 0.0s
=> => unpacking to docker.io/library/lamp_webserver:latest 0.0s
=> => resolving provenance for metadata file 0.0s
[+] Running 4/4
✔ lamp_webserver:latest Built 0.0s
✔ Network nginx_proxy_network Created 0.1s
✔ Container lamp_mariadb Healthy 10.8s
✔ Container lamp_webserver Started 10.9s

```

L'output di docker compose up -d mostra la build dell'immagine lamp_webserver, il pull dell'immagine MariaDB, la creazione della rete e l'avvio condizionato: prima il database raggiunge lo stato "healthy", poi il webserver viene avviato con successo.



La board Kanban traccia le attività dell'elaborato, suddividendo i task tra "Da Fare" e "In Corso": configurazione del Dev Container, build dell'immagine PHP, avvio dello stack LAMP e test delle API, offrendo una chiara panoramica dei passaggi operativi.

```
@eFfe0513 → /workspaces/cloud-computing-lab/docker-container/lamp (main) $ docker compose ps
# Entrambi i container devono essere "Up (healthy)"
NAME                IMAGE              COMMAND                  SERVICE    CREATED         STATUS          PORTS
lamp_mariadb         mariadb:10.11     "docker-entrypoint.s..." mariadb     About a minute ago Up About a minute (healthy) 3306/tcp
lamp_webserver       lamp_webserver:latest "docker-php-entrypoi..." webserver   About a minute ago Up 57 seconds (healthy) 0.0.0.0:8888->80/tcp, [::]:8888->80/tcp
```

L'output di docker compose ps mostra entrambi i container (MariaDB e webserver) in stato "Up (healthy)", confermando che l'intero stack LAMP è correttamente avviato, con il database che ha superato l'healthcheck.

```
@eFfe0513 → /workspaces/cloud-computing-lab/docker-container/lamp (main) $ curl http://localhost:8888/api.php?action=ping
{
  "status": "ok",
  "db": "taskmanager",
  "php": "8.2.5"
}
```

La richiesta curl all'endpoint api.php?action=ping restituisce {"status":"ok","db":"taskmanager","php":"8.2.5"}, dimostrando la connessione funzionante tra il frontend PHP e il database MariaDB.

```

@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $ docker compose down
# I dati MariaDB rimangono nella cartella ./volumes/mysql/
# Per eliminare anche i dati (reset completo):
rm -rf ./volumes/mysql/*
[+] Running 3/3
 ✓ Container lamp_webserver      Removed
 ✓ Container lamp_mariadb        Removed
 ✓ Network nginx_proxy_network  Removed
rm: cannot remove './volumes/mysql/aria_log.00000001': Permission denied
rm: cannot remove './volumes/mysql/aria_log_control': Permission denied
rm: cannot remove './volumes/mysql/ib_buffer_pool': Permission denied
rm: cannot remove './volumes/mysql/ibdata1': Permission denied
rm: cannot remove './volumes/mysql/ib_logfile0': Permission denied
rm: cannot remove './volumes/mysql/multi-master.info': Permission denied
rm: cannot remove './volumes/mysql/mysql': Permission denied
rm: cannot remove './volumes/mysql/mysql_upgrade_info': Permission denied
rm: cannot remove './volumes/mysql/performance_schema': Permission denied
rm: cannot remove './volumes/mysql/sys': Permission denied
rm: cannot remove './volumes/mysql/taskmanager': Permission denied
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $

```

Dopo lo spegnimento dello stack con docker compose down, il tentativo di cancellare i dati persistenti in ./volumes/mysql/ fallisce per alcuni file a causa di permessi insufficienti, segnalando la necessità di elevare i privilegi per una pulizia completa.

```

PROBLEMI  OUTPUT  CONSOLE DI DEBUG  TERMINALE  PORTE
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
d7ec149d5acd   lamp_webserver:latest               "docker-php-entrypoi..." About a minute ago Up About a minute (healthy)  0.0.0.0:8888->80/tcp, [::]:8888->80/tcp   lamp_webserver
4c8e7e14d8c5   mariadb:10.11                       "docker-entrypoint.s..." About a minute ago Up About a minute (healthy)  3306/tcp                                  lamp_mariadb
ffbee0544eb4   spring-dashboard:1.0                "/_cacert_entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  spring-dashboard
124f5b99d92a   nodejs-api:1.0                      "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  nodejs-api
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $

```

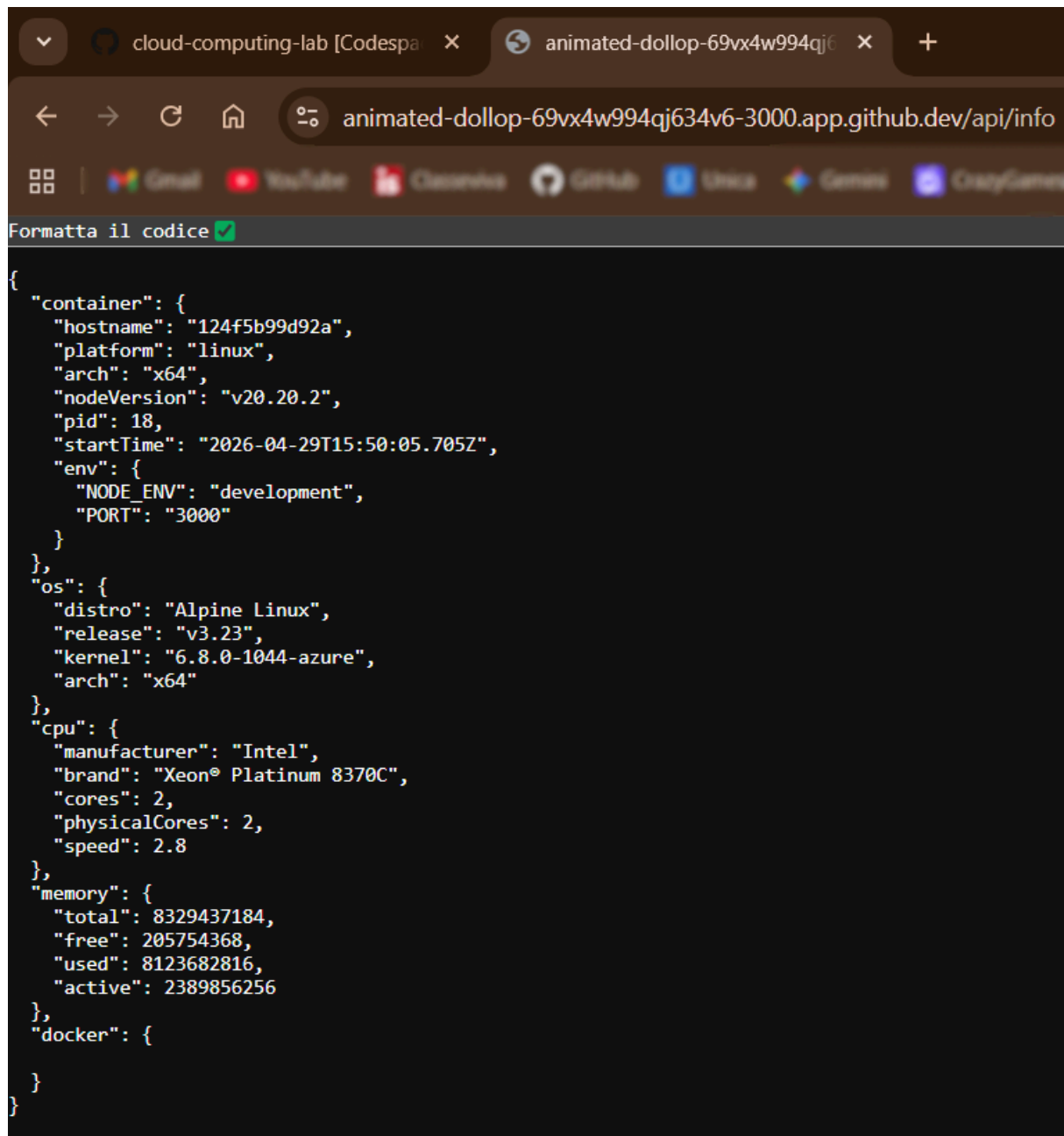
L'output di docker ps elenca i quattro container attivi (Node.js API, Spring Dashboard, MariaDB e webserver LAMP) con le rispettive porte esposte, riepilogando l'ambiente multi-container avviato con successo.

```

@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $ curl http://localhost:8888/api.php?action=ping
{
  "status": "ok",
  "db": "taskmanager",
  "php": "8.2.5"
}
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/lamp (main) $

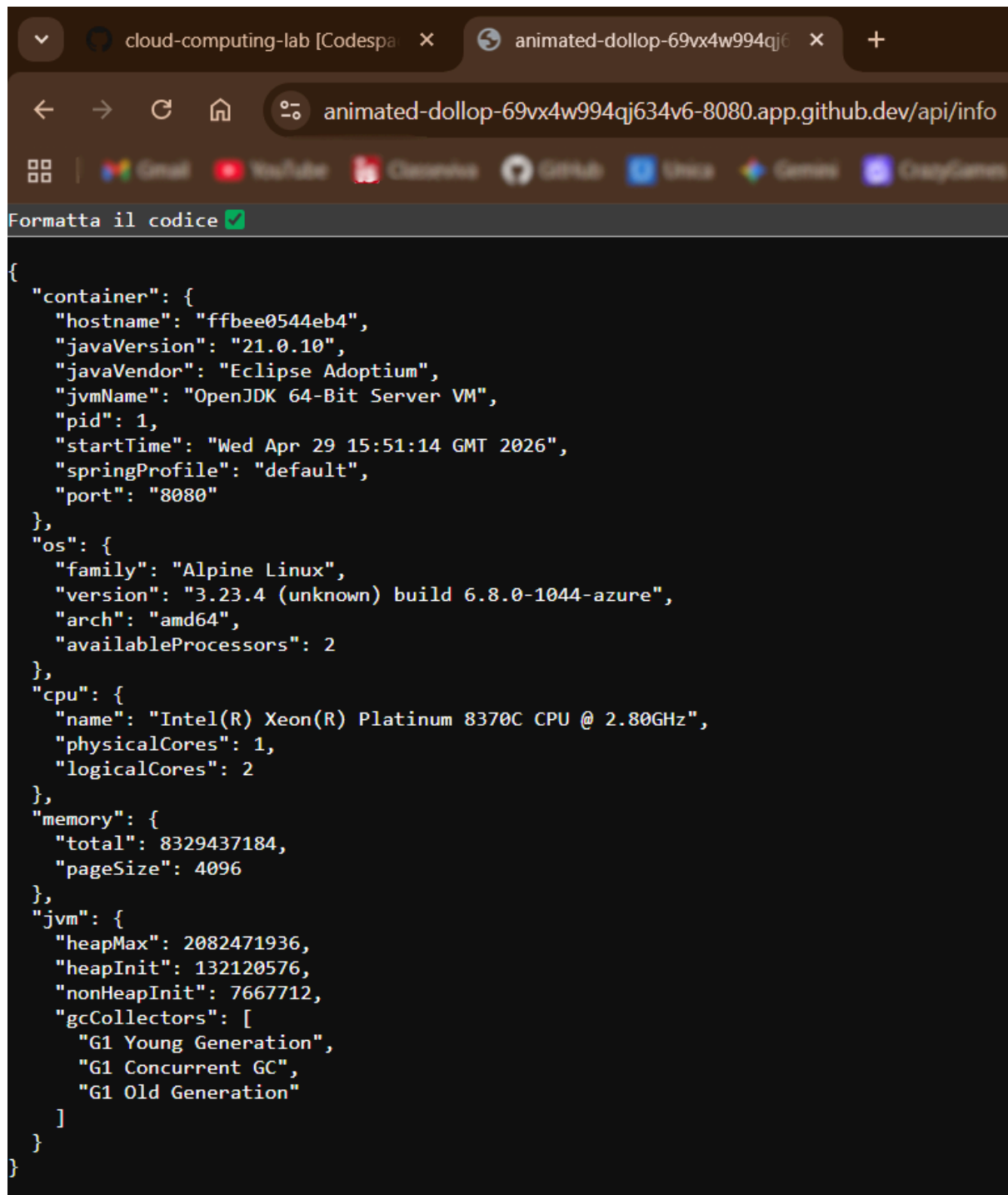
```

Viene nuovamente verificata la connessione tra PHP e MariaDB tramite chiamata all'endpoint api.php?action=ping: la risposta conferma lo stato operativo del database taskmanager e la versione PHP 8.2.5, attestando la stabilità dello stack dopo le precedenti operazioni.



```
{
  "container": {
    "hostname": "124f5b99d92a",
    "platform": "linux",
    "arch": "x64",
    "nodeVersion": "v20.20.2",
    "pid": 18,
    "startTime": "2026-04-29T15:50:05.705Z",
    "env": {
      "NODE_ENV": "development",
      "PORT": "3000"
    }
  },
  "os": {
    "distro": "Alpine Linux",
    "release": "v3.23",
    "kernel": "6.8.0-1044-azure",
    "arch": "x64"
  },
  "cpu": {
    "manufacturer": "Intel",
    "brand": "Xeon® Platinum 8370C",
    "cores": 2,
    "physicalCores": 2,
    "speed": 2.8
  },
  "memory": {
    "total": 8329437184,
    "free": 205754368,
    "used": 8123682816,
    "active": 2389856256
  },
  "docker": {
  }
}
```

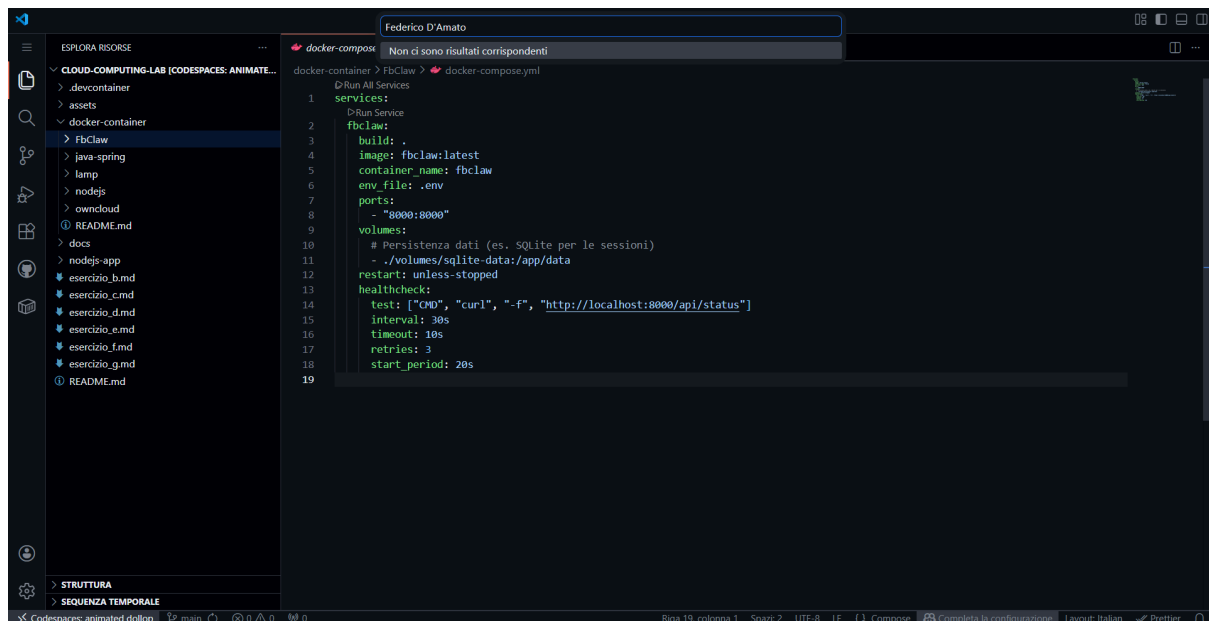
L'API del container Node.js restituisce in formato JSON le informazioni di sistema: hostname, sistema operativo Alpine, CPU Intel Xeon, memoria totale e utilizzata, e le variabili d'ambiente del processo Node, offrendo un quadro aggiornato delle risorse.



The screenshot shows a web browser with two tabs: 'cloud-computing-lab [Codespa' and 'animated-dollop-69vx4w994qj6'. The address bar displays 'animated-dollop-69vx4w994qj634v6-8080.app.github.dev/api/info'. Below the address bar is a toolbar with icons for Gmail, YouTube, Classroom, GitHub, Unica, Gemini, and CrazyGames. A status bar at the top of the content area says 'Formatta il codice' with a green checkmark icon. The main content area displays a JSON object representing container details.

```
{
  "container": {
    "hostname": "ffbee0544eb4",
    "javaVersion": "21.0.10",
    "javaVendor": "Eclipse Adoptium",
    "jvmName": "OpenJDK 64-Bit Server VM",
    "pid": 1,
    "startTime": "Wed Apr 29 15:51:14 GMT 2026",
    "springProfile": "default",
    "port": "8080"
  },
  "os": {
    "family": "Alpine Linux",
    "version": "3.23.4 (unknown) build 6.8.0-1044-azure",
    "arch": "amd64",
    "availableProcessors": 2
  },
  "cpu": {
    "name": "Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80GHz",
    "physicalCores": 1,
    "logicalCores": 2
  },
  "memory": {
    "total": 8329437184,
    "pageSize": 4096
  },
  "jvm": {
    "heapMax": 2082471936,
    "heapInit": 132120576,
    "nonHeapInit": 7667712,
    "gcCollectors": [
      "G1 Young Generation",
      "G1 Concurrent GC",
      "G1 Old Generation"
    ]
  }
}
```

L'endpoint informativo del container Spring Boot espone i dettagli dell'ambiente Java: versione 21.0.10, vendor Eclipse Adoptium, heap massimo, garbage collector G1 e metriche di sistema Alpine, completando il monitoraggio dell'applicazione.



Infine è mostrata una raccolta di comandi docker-compose (up, down, build, logs, ps) e snippet di Dockerfile che affianca la struttura del progetto, fornendo un riepilogo visuale dei principali strumenti CLI utilizzati per orchestrare e mantenere i container.

Procedimento e Risultati

L'elaborato è stato svolto in Codespaces, dopo fork e clone del repository. Per Node.js si è costruita l'immagine nodejs-api:1.0 (da node:20-alpine) e avviato il container sulla porta 3000, verificando la dashboard con metriche di sistema. Per Spring Boot, il Dockerfile multi-stage (Maven/JDK 21 → JRE 21 Alpine) è stato realizzato con successivo avvio sulla porta 8080 e dashboard JVM. Lo stack LAMP, orchestrato con docker compose, ha avviato MariaDB (con healthcheck) e il webserver PHP/Apache; le chiamate API ping e list hanno confermato la connessione al database. Infine, docker ps ha mostrato tutti e quattro i container attivi e docker compose down ha spento i servizi preservando i dati su volume.

Considerazioni Finali

L'attività ha consolidato le competenze di containerizzazione in un ambiente cloud riproducibile, eliminando le complessità locali grazie a Codespaces. L'uso di immagini Alpine e della build multi-stage ha evidenziato l'importanza di ottimizzare le dimensioni delle immagini. Successivamente, l'orchestrazione con docker compose ha introdotto dipendenze, healthcheck e volumi, aspetti chiave in architetture reali. Dunque, i test funzionali con curl e dashboard hanno dato riscontro immediato del corretto funzionamento degli stack. L'esperienza ha rafforzato la comprensione del ciclo di vita dei container, offrendo spunti per approfondire persistenza, rete e sicurezza.

- Esercizio C -

Agente AI con Agno, Mistral e Telegram

Descrizione del Problema

L'esercitazione richiede di realizzare un agente AI nel container `fbclaw` con Agno, Mistral e Groq Whisper, accessibile via web (chat testuale e vocale) e opzionalmente tramite bot Telegram. Si devono configurare le API key necessarie, avviare il container con `docker compose up` e verificare chat, memoria conversazionale, trascrizione audio ed eventuale interazione Telegram.

Implementazione

◆ Laboratorio guidato

Collaudo

```
@eFFe0513 →/workspaces/cloud-computing-lab/docker-container/FbClaw (main) $ docker compose logs -f
fbclaw | INFO: Started server process [1]
fbclaw | INFO: Waiting for application startup.
fbclaw | INFO: Application startup complete.
fbclaw | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
fbclaw | [FbClaw] Bot Telegram avviato in background
fbclaw | [Telegram] Bot avviato, in attesa di messaggi...
fbclaw | INFO: 127.0.0.1:49690 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 172.18.0.1:55748 - "GET / HTTP/1.1" 200 OK
fbclaw | INFO: 172.18.0.1:55752 - "GET /static/app.js HTTP/1.1" 200 OK
fbclaw | INFO: 172.18.0.1:55756 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 172.18.0.1:55766 - "GET /favicon.ico HTTP/1.1" 404 Not Found
fbclaw | INFO: 127.0.0.1:34356 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 127.0.0.1:52314 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 127.0.0.1:50248 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 127.0.0.1:47620 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 127.0.0.1:49578 - "GET /api/status HTTP/1.1" 200 OK
fbclaw | INFO: 127.0.0.1:36718 - "GET /api/status HTTP/1.1" 200 OK
```

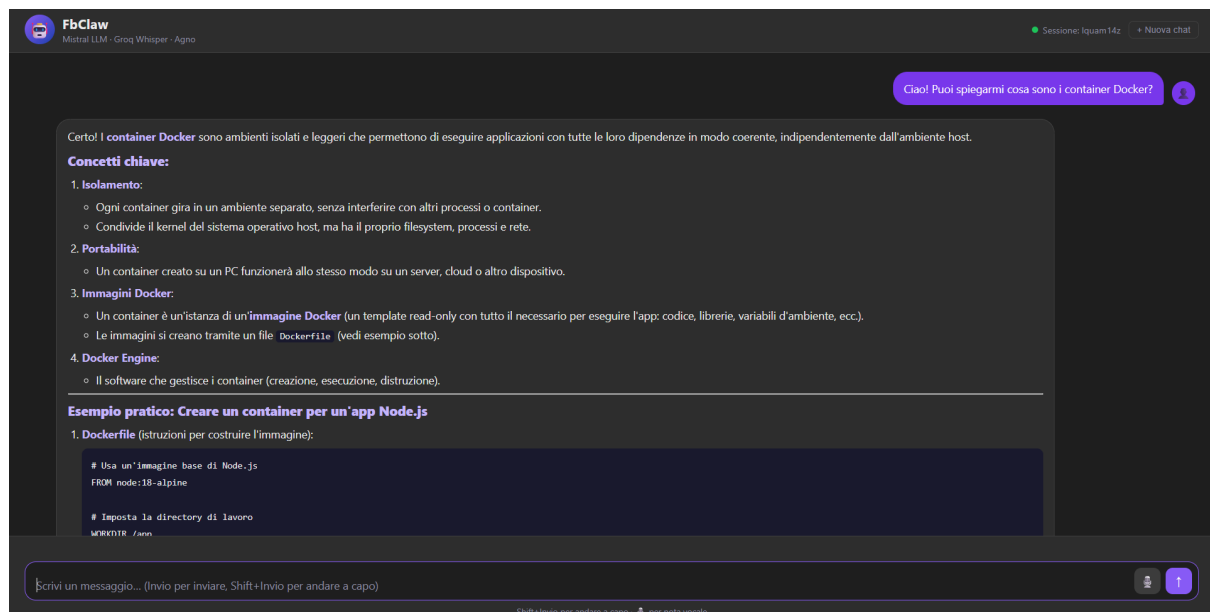
Il log conferma che il server e il bot Telegram sono stati avviati correttamente e risultano ora pienamente operativi e pronti all'uso. Il sistema sta gestendo con successo le frequenti verifiche di stato e il traffico verso l'interfaccia web, confermando che l'applicazione è correttamente funzionante e non presenta criticità.

```

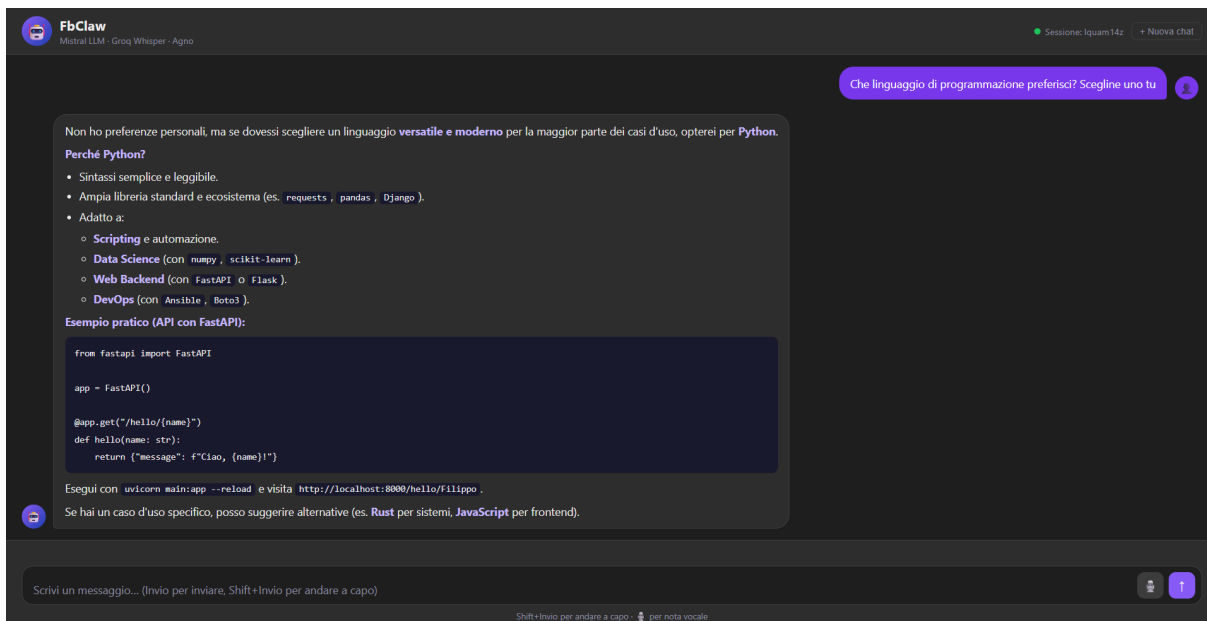
@eFFE0513 → /workspaces/cloud-computing-lab/docker-container/FbClaw (main) $ curl http://localhost:8000/api/status | jq .
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100    90    100    90    0    0  52417      0  --:--:-- --:--:-- --:--:-- 90000
{
  "status": "ok",
  "model": "mistral-large-latest",
  "telegram_enabled": true,
  "active_sessions": 0
}
@eFFE0513 → /workspaces/cloud-computing-lab/docker-container/FbClaw (main) $

```

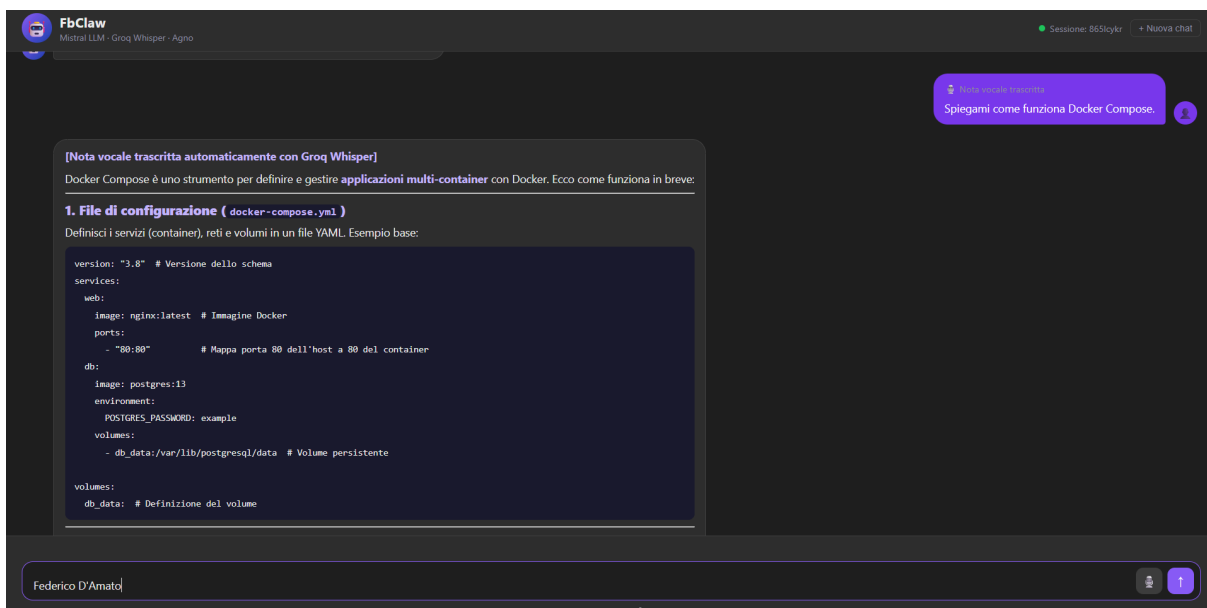
Interrogazione dell'API locale per verificare lo stato del sistema, ricevendo una conferma formale che tutto è correttamente operativo. L'output indica che il bot Telegram è attivo e l'intelligenza artificiale è pronta all'uso, sebbene al momento non risultino ancora sessioni utente in corso.



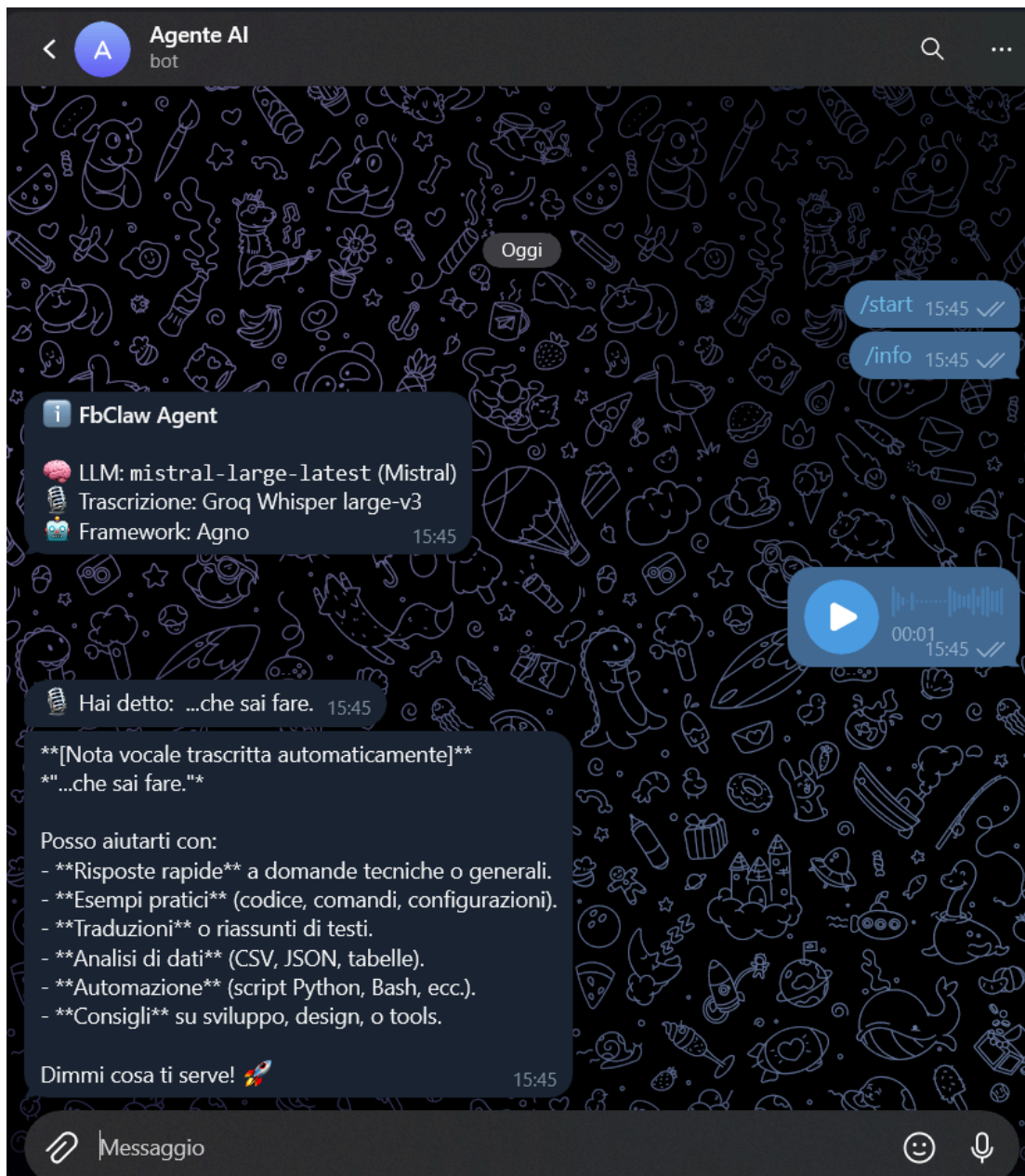
Interfaccia chat dell'applicazione in funzione, con il modello Mistral che risponde correttamente a una domanda tecnica sui container. Questo conferma che l'integrazione tra il frontend e l'intelligenza artificiale è operativa, permettendo al sistema di elaborare e restituire informazioni in modo fluido.



L'assistente fornisce una consulenza tecnica sulla scelta del linguaggio di programmazione, suggerendo Python per la sua versatilità e offrendo esempi pratici di implementazione. L'interazione conferma la capacità del sistema di generare contenuti tecnici coerenti, supportando efficacemente le attività di studio e sviluppo software.



Il sistema sta elaborando un input vocale trascritto automaticamente, fornendo una spiegazione tecnica strutturata su come gestire applicazioni multi-container tramite Docker Compose. L'interazione conferma la corretta integrazione tra i servizi di trascrizione e il modello linguistico per il supporto allo sviluppo software.



Viene mostrata l'interazione finale con il bot Telegram, che conferma di essere operativo e correttamente integrato con le tecnologie Mistral, Groq Whisper e Agno. Il bot ha trascritto con successo un messaggio vocale e ha presentato una lista completa delle proprie funzionalità, dal supporto tecnico all'automazione, dichiarandosi pronto a fornire assistenza immediata.

Procedimento e Risultati

Il procedimento ha previsto la costruzione di un agente AI completo all'interno del container FbClaw, seguendo una serie di passaggi strutturati. Dopo aver ottenuto le chiavi API per Mistral e Groq e aver creato il file .env, il container è stato avviato con il comando `docker compose up -d --build`. Successivamente, una volta verificato il corretto funzionamento, sono stati eseguiti test completi: l'invio di messaggi testuali dall'interfaccia web ha confermato la memoria contestuale dell'agente in

conversazioni multi-turn, mentre la registrazione vocale dal browser è stata correttamente trascritta da Groq Whisper. Il risultato finale è stato un ecosistema integrato funzionante, con un bot Telegram opzionale in esecuzione in background e un agente AI in grado di rispondere sia a comandi testuali che vocali in modo coerente e contestuale.

Domande di riflessione

1. Perché i dati delle sessioni si perdono al riavvio del container? Come si potrebbe aggiungere persistenza (suggerimento: Agno supporta SqliteAgentStorage per salvare le sessioni su file)?

I dati delle sessioni si perdono perché il container usa un filesystem temporaneo che viene cancellato quando si ferma. Per risolvere, è possibile usare SqliteAgentStorage di Agno: basta creare un file .db e installarlo nel container con docker compose, così i dati restano salvati anche dopo il riavvio.

2. Il bot Telegram e il server FastAPI girano nello stesso processo Python. Quali problemi potrebbe causare questa scelta in produzione? Come si separa in due container distinti?

In produzione, se il bot si blocca o va in loop, si ferma anche il server API, e viceversa. Inoltre non è possibile scalare separatamente le due parti. Per separarli, è necessario creare due Dockerfile diversi e due servizi in docker-compose.yml, uno per FastAPI e uno per il bot.

3. Cosa succede se GROQ_API_KEY non è configurata e un utente invia una nota vocale? Come miglioreresti la gestione dell'errore?

Senza la chiave, quando un utente invia una nota vocale, il programma crasha con un errore di configurazione o restituisce un fallimento. Per migliorarlo, è opportuno aggiungere un controllo all'avvio che disabiliti la funzionalità vocale, restituendo un messaggio chiaro del tipo "Servizio vocale non disponibile, contatta l'amministratore".

4. Come implementeresti l'autenticazione per il bot Telegram, limitandone l'accesso solo a certi user_id?

È possibile limitare l'accesso scorrendo una lista di user_id autorizzati da una variabile d'ambiente. Ad ogni messaggio, il bot controlla se il messaggio dall'user è nella lista e, in caso contrario, risponde con "Non sei autorizzato" e ignora il comando. In conclusione, in questo modo solo gli utenti specifici possono interagire.

Considerazioni Finali

L'esercitazione ha dimostrato l'efficacia di Docker come strumento per creare ambienti isolati e riproducibili, riunendo componenti complessi (Agno, Mistral, Whisper) in un'unica soluzione containerizzata. Un aspetto critico emerso è stata la volatilità dei dati, persi al riavvio del container, suggerendo future implementazioni con memoria persistente. Inoltre, l'architettura a processo singolo per FastAPI e il bot Telegram potrebbero richiedere una separazione in container distinti per scenari più produttivi e scalabili, come indicato nelle domande di riflessione fornite.